Abstracción y Polimorfismo: Clases Abstractas en Java

Tecnologías Orientadas a Objetos

Uriel Juárez Martínez

24 de septiembre de 2026

¿Qué es una Clase Abstracta?

Una clase abstracta es una **plantilla estricta** para otras clases.

Regla de Oro:

- No se pueden crear objetos directamente de ella (No se puede instanciar).
- Ejemplo: `new Enemigo()` $\rightarrow$ **ERROR**

¿Para qué sirve?

- Agrupar atributos y comportamientos comunes.
- Dejar los detalles específicos para que las clases hijas los resuelvan.

Anatomía de una Clase Abstracta

Se define usando la palabra reservada `abstract`.
Puede contener dos tipos de métodos:

1. Métodos Concretos

Tienen cuerpo y código ({}). Las clases hijas los heredan tal cual y pueden usarlos inmediatamente.

2. Métodos Abstractos

Solo se definen, no tienen código y terminan en punto y coma (;).
Obligan a las clases hijas a sobreescribirlos (implementarlos).

Ejemplo: Desarrollo de Videojuego (La Plantilla)

Supongamos que estamos programando los enemigos de nuestro juego.

```
// La clase abstracta (Plantilla base)
abstract class Enemigo {
    protected String nombre;
    protected int vida;

    public Enemigo(String nombre, int vida) {
        this.nombre = nombre;
        this.vida = vida;
    }

    // Metodo CONCRETO: Todos reciben dano igual
    public void recibirDano(int dano) {
        this.vida -= dano;
        System.out.println(nombre + " recibe " + dano + " de dano."
    );
    }

    // Metodo ABSTRACTO: Cada enemigo ataca diferente
    public abstract void atacar();
}
```

Ejemplo: Desarrollo de Videojuego (Las Hijas)

Las clases hijas resuelven cómo ataca cada enemigo.

```
class Zombie extends Enemigo {
    public Zombie() {
        super("Zombie Caminante", 50);
    }
    // OBLIGADO a implementar atacar()
    @Override
    public void atacar() {
        System.out.println(nombre + " muerde. Causa 10 de dano.");
    }
}

class JefeFinal extends Enemigo {
    public JefeFinal() {
        super("Rey Exanime", 500);
    }
    @Override
    public void atacar() {
        System.out.println(nombre + " lanza fuego. Causa 80 de dano
    !");
    }
}
```

Ejercicio Práctico: Sistema de Empleados

Objetivo: Crear una jerarquía de clases usando abstracción.

1. Creen una clase abstracta `Empleado` con atributos *nombre* e *id*.
2. Crear un método concreto `mostrarDatos()`.
3. Crear un método abstracto `calcularSalario()` que devuelva un `double`.
4. Crear clase hija `EmpleadoAsalariado`:
   - Atributo: *salarioMensual*.
   - `calcularSalario()` retorna el salario mensual.
5. Crear clase hija `EmpleadoPorHora`:
   - Atributos: *horasTrabajadas*, *pagoPorHora*.
   - `calcularSalario()` retorna la multiplicación de ambas.

Solución al Ejercicio

```java
abstract class Empleado {
    protected String nombre;
    protected String id;

    public Empleado(String nombre, String id) {
        this.nombre = nombre; this.id = id;
    }
    public void mostrarDatos() {
        System.out.println("Empleado: " + nombre + " | ID: " + id);
    }
    public abstract double calcularSalario(); // Sin cuerpo
}

class EmpleadoPorHora extends Empleado {
    private int horas, pago;
    public EmpleadoPorHora(String n, String id, int h, int p) {
        super(n, id); this.horas = h; this.pago = p;
    }
    @Override
    public double calcularSalario() {
        return horas * pago;
    }
}
```

¡A programar!

Abran su IDE e implementen el ejercicio.